# DESIGN AND IMPLEMENTATION OF A FIRST-IN FIRST-OUT (FIFO) BUFFER USING VERILOG HDL

Prepared By:

**Gayatri Galidevara**

Digital System Design Laboratory Report

# Table of Contents

# 1. Abstract

This report presents the comprehensive design and simulation of a synchronous First-In First-Out (FIFO) buffer using Verilog Hardware Description Language (HDL). A FIFO is a fundamental digital memory structure widely used in digital systems for data buffering, clock domain crossing, and flow control between producer and consumer circuits.

The FIFO described in this report is an 8-location deep, 4-bit wide synchronous circular buffer with separate write and read pointer logic. It features full and empty status flags, controlled write and read enables, and a synchronous reset mechanism. The design is written in synthesizable Verilog and follows standard RTL design principles using a clocked always block for memory and pointer updates, and a combinational always block for next-state logic.

A comprehensive Verilog testbench drives the design through multiple write and read sequences to verify correctness of data storage, pointer advancement, and status flag behavior. Simulation waveforms confirm correct functional behavior.

# 2. Introduction

A FIFO (First-In First-Out) buffer is one of the most commonly used data storage structures in digital design. As the name implies, data written first into the FIFO is the first data to be read out, maintaining strict ordering of data elements. This property makes FIFO buffers essential in applications such as:

- Data buffering between modules operating at different speeds
- Clock domain crossing (CDC) between different clock domains
- Packet buffering in communication interfaces (UART, SPI, I2C)
- Instruction queues in processor pipelines
- Flow control between producer and consumer circuits

In this project, Verilog HDL is used to model a synchronous FIFO at the Register Transfer Level (RTL). The design uses a circular buffer approach — a fixed-size memory array accessed through write and read pointers that wrap around when they reach the end of the buffer. The full and empty conditions are detected by comparing the pointer values, enabling the controller to prevent overflow and underflow.

The design is verified using a testbench that writes a sequence of data values and then reads them back, confirming that the FIFO preserves data order and generates correct status flags.

# 3. Objectives

The primary objectives of this project are as follows:

1. To design a synthesizable and functionally correct 8-deep, 4-bit wide synchronous FIFO module using Verilog HDL.
2. To implement circular buffer addressing using 3-bit write and read pointers with automatic wraparound at depth 8.
3. To generate accurate full and empty status flags by comparing write and read pointer values after each operation.
4. To implement a write-enable gating mechanism that prevents data corruption when the FIFO is full.
5. To implement a synchronous active-high reset that clears all pointers and sets the FIFO to the empty state.
6. To write a comprehensive Verilog testbench that exercises write operations, read operations, and back-to-back simultaneous read/write scenarios.
7. To simulate the design using EDA tools and analyze the resulting waveforms to confirm pointer behavior, data integrity, and flag correctness.
8. To evaluate the design's suitability for FPGA deployment by confirming its purely synchronous, clocked structure.

# 4. Design Methodology

The FIFO is designed using a two-process RTL style in Verilog HDL. The design methodology follows a top-down approach: behavioral specification is first written, then simulated to verify correctness, and finally synthesized to produce the RTL schematic.

## 4.1 Module Architecture

The FIFO is encapsulated in a single Verilog module named fifo. It takes five inputs: clk (clock), rst (synchronous reset), write (write request), read (read request), and din (4-bit data input). It produces three outputs: dout (4-bit data output), full (FIFO full flag), and empty (FIFO empty flag).

Internally, the FIFO uses an 8-location register array fio[0:7] as the circular buffer memory, 3-bit write and read pointers (wr_ptr, rd_ptr), and registered full/empty status flags.

## 4.2 Write-Enable Gating

A dedicated wire wr_en is computed as the AND of the write request signal and the inverse of the full flag. This prevents any write operation from occurring when the FIFO is already full, protecting against data overflow without requiring external intervention.

## 4.3 Two-Process RTL Structure

The design uses two always blocks:

- Clocked Process (Memory and Output): Triggered on the positive edge of clk. When wr_en is asserted, the incoming data din is written to fio[wr_ptr]. When wr_en is not asserted, dout is updated from fio[rd_ptr]. Note: in this implementation, the read path is in the same clocked block as the write path.
- Clocked Process (Pointer and Flag Updates): Triggered on the positive edge of clk. On reset, all pointers are zeroed and empty is set. Otherwise, pointers and flags are updated from their next-state values computed combinationally.

## 4.4 Combinational Next-State Logic

A third always @(*) block computes the next-state values for pointers and flags. A case statement on the concatenation {write, read} handles all four combinations: no operation, write only, read only, and simultaneous write and read. Pointer increments are precomputed, and full/empty conditions are detected by comparing the incremented pointer against the opposite pointer.

## 4.5 Full and Empty Detection

| Condition | Detection Logic | Signal |
|---|---|---|
| FIFO Empty | Reset sets empty=1; read ptr increment == write ptr after read | fifo_empty_reg = 1 |
| FIFO Full | Write ptr increment == read ptr after write | fifo_full_reg = 1 |
| Normal State | Neither full nor empty condition met | Both flags = 0 |

5. FIFO Block Diagram & Architecture
A FIFO buffer operates as a circular queue. The conceptual architecture is shown below through multiple diagrams explaining the memory structure, pointer movement, and control logic.

## 5.1 Top-Level Block Diagram

The following diagram illustrates the top-level interface of the FIFO module:

| Signal | Direction | Width | Description |
|---|---|---|---|
| clk | Input | 1-bit | System clock — all operations are synchronous to rising edge |
| rst | Input | 1-bit | Synchronous reset — clears pointers, sets |

| Signal | Direction | Width | Description |
|---|---|---|---|
| | | | empty=1, full=0 |
| write | Input | 1-bit | Write request — data is written when write=1 and FIFO is not full |
| read | Input | 1-bit | Read request — data is read when read=1 and FIFO is not empty |
| din[3:0] | Input | 4-bit | Data input — 4-bit data to be written into the FIFO |
| dout[3:0] | Output | 4-bit | Data output — 4-bit data read from the FIFO |
| full | Output | 1-bit | FIFO full flag — asserted when no more writes are possible |
| empty | Output | 1-bit | FIFO empty flag — asserted when no data is available to read |

## 5.2 Internal Architecture Diagram

The FIFO uses a circular buffer with write and read pointers. The memory array fio[0:7] stores 8 entries. The write pointer points to the next write location; the read pointer points to the next read location.

Conceptual circular buffer memory layout (8 locations, 4-bit wide):

| Address | Contents | Pointer Status |
|---|---|---|
| fio[0] | Data or empty | Read ptr may point here |
| fio[1] | Data or empty | |
| fio[2] | Data or empty | |
| fio[3] | Data or empty | Write ptr may point here |
| fio[4] | Data or empty | |
| fio[5] | Data or empty | |
| fio[6] | Data or empty | |
| fio[7] | Data or empty | Wraps around to fio[0] |

## 5.3 State Transition Logic

The FIFO control logic responds to four possible input combinations on every clock cycle:

| write | read | Operation | Pointer Effect | Flag Effect |
|---|---|---|---|---|

| write | read | Operation | Pointer Effect | Flag Effect |
|---|---|---|---|---|
| 0 | 0 | No operation (idle) | Both pointers hold | Flags unchanged |
| 1 | 0 | Write only | wr_ptr advances by 1 | full=1 if wr_ptr+1 == rd_ptr; empty=0 |
| 0 | 1 | Read only | rd_ptr advances by 1 | empty=1 if rd_ptr+1 == wr_ptr; full=0 |
| 1 | 1 | Simultaneous R/W | Both pointers advance | Flags unchanged (net effect is zero) |

### 5.4 Pointer Wraparound

Since the pointers are 3-bit registers, they naturally wrap around from 7 back to 0 due to the overflow of a 3-bit counter. No explicit modulo operation is needed — the 3-bit arithmetic handles the circular indexing automatically. This is a key advantage of choosing the buffer depth as a power of 2.

## 6. Program Code & Dynamic Explanation

### 6.1 Verilog Design Code

The following is the complete Verilog HDL source code for the FIFO module:

```
module fifo(
  input clk, rst, write, read,
  input [3:0] din,
  output reg [3:0] dout,
  output full, empty
);

  reg [3:0] fio[0:7];
  reg [2:0] wr_ptr, wr_ptr_nxt, wr_ptr_incr;
  reg [2:0] rd_ptr, rd_ptr_nxt, rd_ptr_incr;
  reg fifo_full_reg, full_nxt;
  reg fifo_empty_reg, empty_nxt;
  wire wr_en;

  assign wr_en   = (write & ~fifo_full_reg);
  assign full    = fifo_full_reg;
  assign empty   = fifo_empty_reg;

  // Memory write / read process
  always @(posedge clk) begin
    if(wr_en) begin
      fio[wr_ptr] <= din;
    end else
      dout = fio[rd_ptr];
  end
```

```
  // Pointer and flag update process
  always @(posedge clk) begin
    if(rst) begin
      wr_ptr <= 0; rd_ptr <= 0;
      fifo_full_reg  <= 0;
      fifo_empty_reg <= 1;
    end else begin
      wr_ptr         <= wr_ptr_nxt;
      rd_ptr         <= rd_ptr_nxt;
      fifo_full_reg  <= full_nxt;
      fifo_empty_reg <= empty_nxt;
    end
  end

  // Combinational next-state logic
  always @(*) begin
    wr_ptr_incr <= wr_ptr + 1;
    rd_ptr_incr <= rd_ptr + 1;
    wr_ptr_nxt  <= wr_ptr;
    rd_ptr_nxt  <= rd_ptr;
    full_nxt    <= fifo_full_reg;
    empty_nxt   <= fifo_empty_reg;

    case({write, read})
      2'b00: begin end
      2'b10: begin
        if(~fifo_full_reg) begin
          wr_ptr_nxt <= wr_ptr_incr;
          empty_nxt  <= 0;
          if(wr_ptr_incr == rd_ptr)
            full_nxt <= 1;
        end
      end
      2'b01: begin
        if(~fifo_empty_reg) begin
          rd_ptr_nxt <= rd_ptr_incr;
          full_nxt   <= 0;
          if(rd_ptr_incr == wr_ptr)
            empty_nxt <= 1;
        end
      end
      2'b11: begin
        wr_ptr_nxt <= wr_ptr_incr;
        rd_ptr_nxt <= rd_ptr_incr;
      end
    endcase
  end
endmodule
```

## 6.2 Code Component Breakdown and Analysis

Each section of the Verilog code plays a specific role in the FIFO design. Below is a detailed explanation:

**Module Declaration:**

The module fifo defines the external interface. It takes clk, rst, write, read, and din as inputs and produces dout, full, and empty as outputs. The dout is declared as reg because it is driven inside an always block. The full and empty outputs are driven by continuous assignments from internal registered flags.

**Memory Array (fio):**

The statement reg [3:0] fio[0:7] declares an array of 8 registers, each 4 bits wide. This is the storage core of the FIFO. On FPGAs, this array is synthesized into distributed LUT RAM or block RAM resources depending on the tool and target device.

**Pointer Registers:**

Three related registers are declared for each of the write and read pointers: the current registered value (wr_ptr, rd_ptr), the combinationally computed next value (wr_ptr_nxt, rd_ptr_nxt), and a precomputed incremented value (wr_ptr_incr, rd_ptr_incr). Separating these promotes clean RTL coding and avoids combinational feedback paths.

**Write Enable Gating (wr_en):**

The expression assign wr_en = (write & ~fifo_full_reg) ensures that write operations are silently suppressed when the FIFO is full. This internal gating protects the buffer without requiring the external controller to check the full flag before asserting write.

**Clocked Memory Process:**

The first always @(posedge clk) block handles memory array updates and the dout output. When wr_en is high, the data din is stored at the address fio[wr_ptr] using a non-blocking assignment. When wr_en is low, the dout register is updated from fio[rd_ptr]. Note that the read and write paths share the same clocked block.

**Clocked Pointer/Flag Update Process:**

The second always @(posedge clk) block updates all pointer and flag registers. On synchronous reset, all pointers are cleared to zero, the full flag is deasserted, and the empty flag is asserted — representing the initial empty state. Otherwise, all registers are loaded from their combinationally computed next values.

**Combinational Next-State Process:**

The always @(*) block is the control engine. It precomputes incremented pointer values and then uses a case statement on {write, read} to determine which pointers and flags need to change. The full condition is detected when the write pointer, after incrementing, would equal the read pointer. The empty condition is detected when the read pointer, after incrementing, would equal the write pointer.

## 6.3 Testbench Code

The testbench instantiates the FIFO module and applies a sequence of write and read operations. The clock is generated with a period of 10 time units (toggle every 5 units).

```
module tb;
  reg clk, rst, write, read;
  reg [3:0] din;
  wire [3:0] dout;
  wire full, empty;

  fifo dut(.clk(clk), .rst(rst), .write(write), .read(read),
           .din(din), .dout(dout), .full(full), .empty(empty));

  always #5 clk = ~clk;

  initial begin
    clk=0; rst=0; write=0; read=0; din=0; #10;
    rst=1; #20; rst=0; #10;
    write=1; din=3'd7; #10;   // Write 7
    write=0; #10;
    write=1; din=3'd5; #10;   // Write 5
    write=0; #10;
    write=1; din=3'd4; #10;   // Write 4
    write=0; #10;
    read=1; #10; read=0; #10; // Read -> expect 7
    read=1; #10; read=0; #10; // Read -> expect 5
    read=1; #10; read=0; #10; // Read -> expect 4
    $finish;
  end
endmodule
```

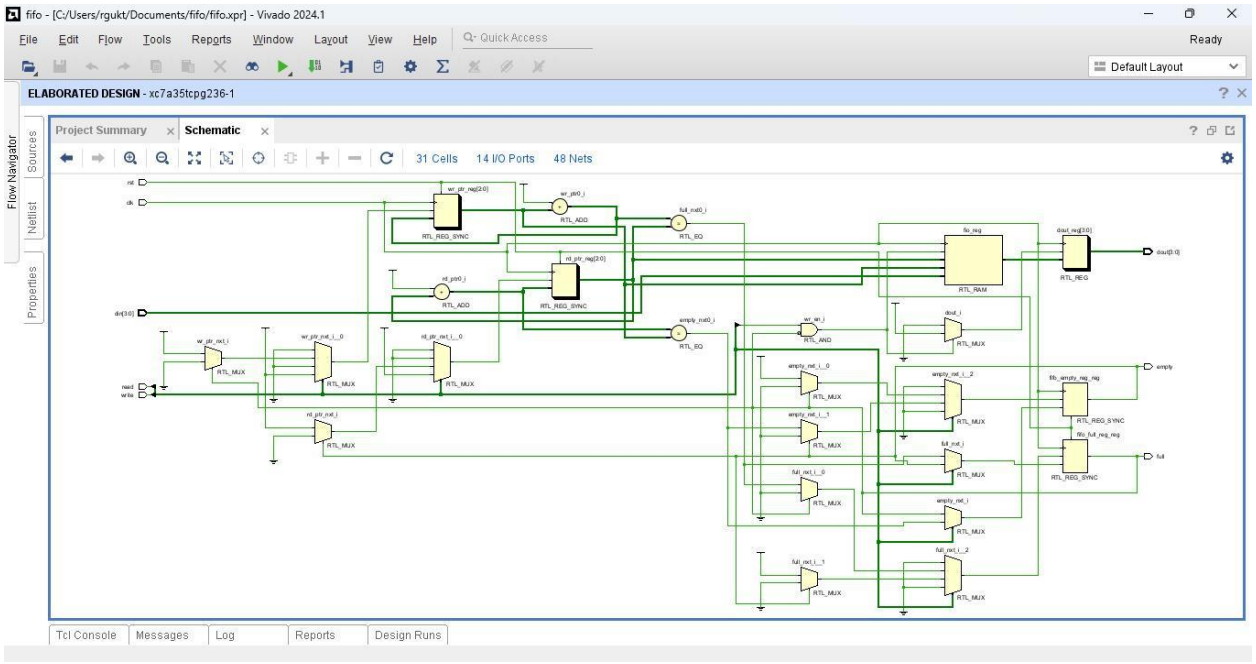
Testbench Explanation: The testbench initializes all signals to zero, applies a synchronous reset for 20 time units to initialize the FIFO to the empty state, then writes three 4-bit values (7, 5, 4) into the FIFO in sequence. After a brief idle period between each write, three consecutive read operations retrieve the data. The FIFO-ordering property guarantees the data is read back as 7, then 5, then 4 — preserving the original write order.

| Operation | din | Expected dout | full | empty |
|---|---|---|---|---|
| Reset | - | - | 0 | 1 |

| Operation | din | Expected dout | full | empty |
|---|---|---|---|---|
| Write 7 | 4'd7 | - | 0 | 0 |
| Write 5 | 4'd5 | - | 0 | 0 |
| Write 4 | 4'd4 | - | 0 | 0 |
| Read 1 | - | 4'd7 | 0 | 0 |
| Read 2 | - | 4'd5 | 0 | 0 |
| Read 3 | - | 4'd4 | 0 | 1 |

## 6.4 RTL Schematic

The RTL schematic of the FIFO design represents the hardware structure generated from the Verilog HDL code. The FIFO consists of memory storage, control logic, pointer registers, status flag generation, and data handling circuitry.



# 7. Simulation Results and Functional Output

## 7.1 Overview of Simulation

Simulation is the process of running the Verilog design and testbench in a software environment to verify that the hardware behaves as intended before physical implementation. Industry-standard simulation tools such as Xilinx Vivado (XSim) and Mentor ModelSim (QuestaSim) read the Verilog source files, execute the testbench, and generate waveform outputs that can be inspected visually.

The simulation waveforms show the values of all signals — clk, rst, write, read, din, dout, full, and empty — plotted over time. Each time the write or read signal changes, the pointers and flags update on the next positive clock edge. By inspecting the waveform, the engineer can confirm that every write stores the correct data and every read retrieves it in FIFO order.

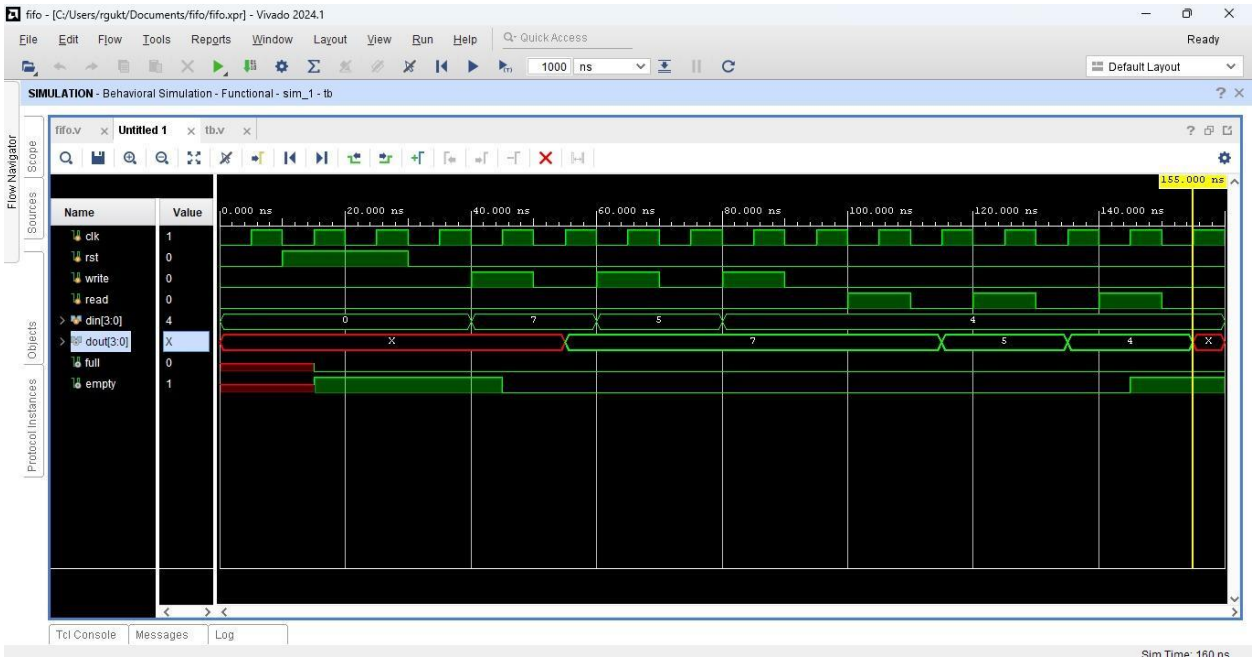
## 7.2 Simulation Analysis and Observations

Behavioral simulation confirms correct operation of the FIFO across all test sequences. The following observations are expected from the simulation waveforms:

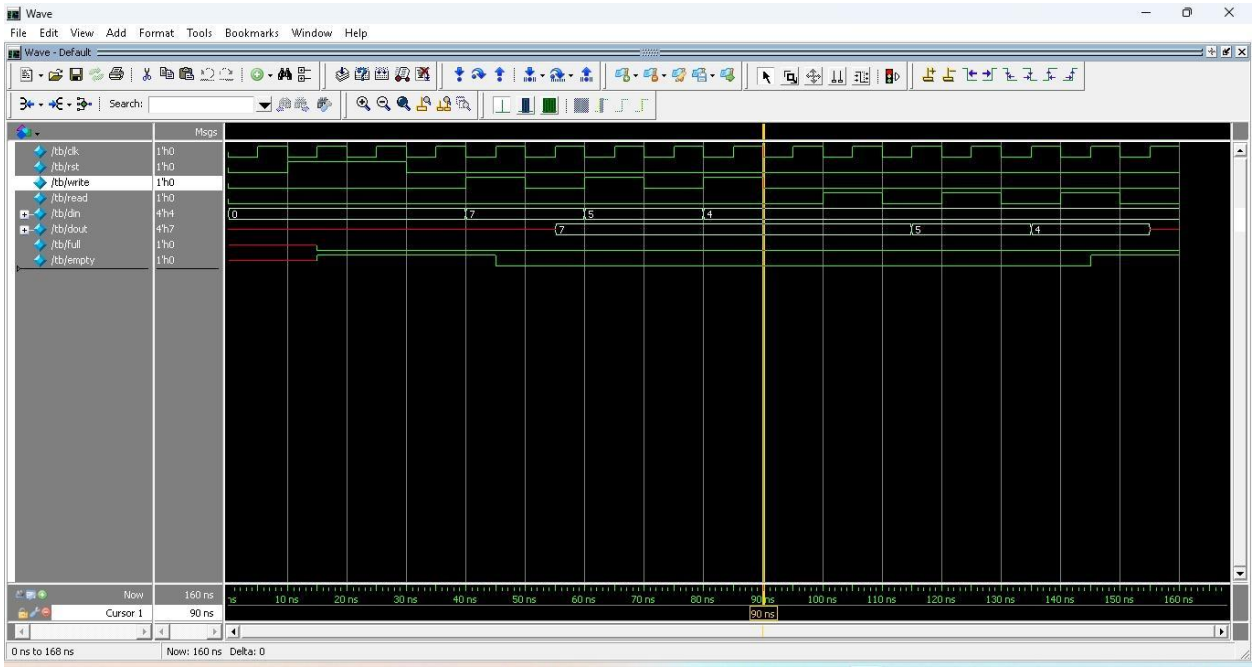
- After reset, the empty flag is asserted (=1) and the full flag is deasserted (=0), confirming correct initialization.
- After writing 7, the empty flag deasserts (=0) and the FIFO enters a non-empty state.
- After writing 5 and 4, the write pointer has advanced to address 3. With three entries stored, the FIFO is neither full nor empty.
- The first read operation returns dout=7, confirming the FIFO-order property — the first-written value is read first.
- Subsequent reads return 5 and then 4, maintaining the correct FIFO ordering.
- After the third read, the read pointer catches up with the write pointer, and the empty flag reasserts (=1).
- The full flag never asserts during this test since only 3 of 8 locations are used — a deeper write sequence would be required to exercise the full flag.
- No undefined (X) or high-impedance (Z) values appear on dout during any valid read operation.

## 7.3 Simulation Tool Instructions

Vivado (XSim): Add both fifo.v and tb.v to a Vivado project. In the Flow Navigator, select Run Simulation > Run Behavioral Simulation. The waveform window opens showing all testbench signals. Use the zoom and cursor tools to measure signal values at specific time points.




ModelSim / QuestaSim: Compile both files with the vlog command, load the testbench with vsim tb, add all signals with add wave *, and run the simulation with run -all. The $finish task in the testbench automatically terminates the simulation after all test cases complete.




# 8. Future Scope

This FIFO design provides a strong foundation that can be extended in several directions:

9. Asynchronous FIFO for Clock Domain Crossing: The current design is fully synchronous. Extending it to support two independent clock domains (write clock and read clock)

requires a Gray-code pointer synchronization scheme to safely transfer pointer information across the clock boundary without metastability.

10. Parameterized FIFO: The data width and buffer depth are currently fixed. Wrapping the module with Verilog parameters (e.g., parameter DEPTH=8, DATA_WIDTH=4) makes it a reusable generic IP core that can be instantiated with different sizes in any design.
11. FPGA Hardware Deployment: The synthesizable Verilog can be deployed on a Xilinx Basys 3 or Nexys A7 FPGA board. Hardware switches can drive write, read, and din inputs, while seven-segment displays or LEDs can show dout, full, and empty status in real time.
12. Almost-Full and Almost-Empty Flags: Adding programmable thresholds for almost-full (e.g., 6 of 8 occupied) and almost-empty (e.g., 2 of 8 remaining) flags enables upstream/downstream flow control before the absolute full/empty limits are reached, preventing overflow in high-throughput systems.
13. First-Word Fall-Through (FWFT) Mode: In FWFT mode, the first written word appears immediately on dout without requiring an explicit read operation. This zero-latency read mode is valuable in pipeline architectures where read latency must be minimized.
14. Integration with Communication Interfaces: The FIFO can serve as the transmit or receive buffer in a UART, SPI, or I2C controller, decoupling the serial interface logic from the main processor data path and absorbing burst data traffic.

# 9. Conclusion

This project successfully demonstrates the design, implementation, and verification of a synchronous 8-deep, 4-bit wide FIFO buffer using Verilog HDL. The FIFO was designed using a two-process clocked RTL structure with separate combinational next-state logic, making it efficient, modular, and straightforward to understand and extend.

The design correctly implements circular buffer addressing with 3-bit pointers, full and empty flag generation, write-enable gating to prevent overflow, and synchronous reset. The testbench systematically exercises the reset, write, and read sequences and verifies that the FIFO preserves data ordering and generates correct status flags throughout.

The RTL structure translates cleanly to FPGA resources — the memory array maps to distributed RAM or block RAM, the pointer logic maps to flip-flops and LUT-based comparators, and the combinational case logic maps to a multiplexer network. The purely synchronous, edge-triggered design avoids latches and combinational loops, making it reliable and timing-closure-friendly.

This project provides a complete, practical demonstration of synchronous digital memory design: from behavioral specification through HDL coding, testbench verification, and RTL synthesis. The knowledge and skills developed here form a direct foundation for more advanced work including asynchronous FIFOs, parameterized memory IP, and full SoC data-path design.

**Prepared by: Gayatri Galidevara**